poll() - Scalable I/O Multiplexing Without FD Limits

Theory, Edge Cases & Code Examples

- ✓ The Problem with select()
- ✓ What is poll()?
- ✓ The pollfd Structure
- ✓ Step by Step
- ✓ Detailed Comparison

→ Swipe to learn →

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 The Problem with select()

In Topic 3, we learned select() for I/O multiplexing. However, it has critical limitations:

select() limitations:

```
// 1. FD_SETSIZE limit (1024 FDs)
if (new_fd >= FD_SETSIZE) {  // ❌ Must reject
    close(new_fd);
}

// 2. Must copy fd_set every iteration
read_fds = master_fds;  // ❌ Expensive for large sets

// 3. Must track max_fd manually
if (new_fd > max_fd) max_fd = new_fd;

// 4. O(max_fd) complexity even with sparse FDs
for (int fd = 0; fd <= max_fd; fd++) {  // ❌ Checks all FDs
    if (FD_ISSET(fd, &read_fds)) {
        // ...
    }
}
```

Real-world impact:
- Can't handle more than 1024 connections
- Inefficient with high FD numbers (e.g., FD 5000 with only 10 active)
- API awkward (bitmask manipulation)

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 What is poll()?

`poll()` is a POSIX system call (since POSIX.1-2001) that solves select()'s FD_SETSIZE limitation while maintaining similar functionality.

Function signature:

```
#include <poll.h>

int poll(struct pollfd *fds,      // Array of file descriptors
         nfds_t nfds,              // Number of elements in fds
         int timeout);             // Timeout in milliseconds

// Returns: number of ready FDs, 0 on timeout, -1 on error
```

Key improvements over select():

1. **No FD_SETSIZE limit**: Can monitor millions of FDs (only limited by system resources)
2. **No max_fd tracking**: Pass array size directly
3. **No bitmask copying**: Input (events) and output (revents) are separate
4. **Better API**: Array of structs instead of opaque bitmasks

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 The pollfd Structure

```
struct pollfd {
    int fd;           // File descriptor
    short events;     // Events to monitor (input)
    short revents;    // Events that occurred (output)
};
```

Event flags:

Flag	Direction	Meaning
`POLLIN`	Input	Data available for reading
`POLLOUT`	Input	Can write without blocking
`POLLPRI`	Input	High-priority data available
`POLLERR`	Output	Error condition
`POLLHUP`	Output	Hang up (peer closed)
`POLLNVAL`	Output	Invalid request (FD not open)

Key insight: `events` is what **you want** to monitor, `revents` is what **actually happened**.

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 How poll() Works - Step by Step (1/2)

Basic example:

```
// Setup: Create array of pollfd structures
std::vector<struct pollfd> fds;

// Add server socket
fds.push_back({server_fd, POLLIN, 0});

// Add clients
fds.push_back({client1_fd, POLLIN, 0});
fds.push_back({client2_fd, POLLIN, 0});

// Call poll()
int ready = poll(fds.data(), fds.size(), -1);  // -1 = block forever

if (ready > 0) {
    // Check which FDs are ready
    for (auto& pfd : fds) {
        if (pfd.revents & POLLIN) {
            // This FD has data to read
            if (pfd.fd == server_fd) {
                // New connection
            } else {
                // Client data
            }
        }

        if (pfd.revents & POLLHUP) {
            // Client disconnected
        }

        if (pfd.revents & POLLERR) {
            // Error on this FD
        }
    }
}
```

Key differences from select():

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 How poll() Works - Step by Step (2/2)

Aspect	select()	poll()
Data structure	fd_set (bitmask)	struct pollfd array
Input/output	Same fd_set (modified)	Separate events/revents
FD limit	1024 (FD_SETSIZE)	Unlimited
Tracking max	Manual max_fd	Array size
Sparse FDs	Still O(max_fd)	O(num_fds)

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 poll() vs select() - Detailed Comparison (1/2)

API complexity:

```
// select(): Complex setup
fd_set master_fds, read_fds;
FD_ZERO(&master_fds);
FD_SET(server_fd, &master_fds);
FD_SET(client_fd, &master_fds);
int max_fd = std::max(server_fd, client_fd);

read_fds = master_fds;  // Copy every iteration
select(max_fd + 1, &read_fds, NULL, NULL, NULL);

for (int fd = 0; fd <= max_fd; fd++) {
    if (FD_ISSET(fd, &read_fds)) {
        // Handle fd
    }
}

// poll(): Simpler
std::vector<struct pollfd> fds = {
    {server_fd, POLLIN, 0},
    {client_fd, POLLIN, 0}
};

poll(fds.data(), fds.size(), -1);

for (auto& pfd : fds) {
    if (pfd.revents & POLLIN) {
        // Handle pfd.fd
    }
}
```

Adding/removing FDs:

```
// select(): Complex
FD_SET(new_fd, &master_fds);
if (new_fd > max_fd) max_fd = new_fd;

FD_CLR(old_fd, &master_fds);
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 poll() vs select() - Detailed Comparison (2/2)

[Code continues across slides]

```
// ...continued from previous slide
// max_fd might be stale

// poll(): Simple
fds.push_back({new_fd, POLLIN, 0});

fds.erase(std::remove_if(fds.begin(), fds.end(),
    [fd](const pollfd& p) { return p.fd == fd; }), fds.end());
```

No FD_SETSIZE limit:

```
// select(): Must reject high FD numbers
if (new_fd >= FD_SETSIZE) {
    close(new_fd);  // Can't handle
}

// poll(): No such limit
fds.push_back({new_fd, POLLIN, 0});  // Works for any FD number
```

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Timeout Handling in poll()

Unlike select()'s `struct timeval`, poll() uses **milliseconds**.

```
// Three timeout modes:

// 1. Block forever
int ready = poll(fds.data(), fds.size(), -1);

// 2. Non-blocking poll (return immediately)
int ready = poll(fds.data(), fds.size(), 0);

// 3. Timeout in milliseconds
int ready = poll(fds.data(), fds.size(), 5000);  // 5 seconds

if (ready == 0) {
    // Timeout expired
}
```

Comparison with select():

Operation	select()	poll()
Block forever	NULL	-1
Non-blocking	{0, 0}	0
5 seconds	{5, 0}	5000
500 ms	{0, 500000}	500

Note: poll() timeout is simpler (single integer) but less precise than select()'s microsecond resolution.

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Event Flags in Detail

Input events (set in events field):

```
struct pollfd pfd;
pfd.fd = client_fd;
pfd.events = POLLIN | POLLOUT;  // Monitor read AND write
pfd.revents = 0;
```

POLLIN - Data available for reading:
- Listening socket: New connection available
- Connected socket: Data ready to read
- EOF received: Also triggers POLLIN (recv returns 0)

POLLOUT - Can write without blocking:
- Socket send buffer has space
- Useful for handling partial sends

POLLPRI - High-priority data:
- TCP urgent data (OOB)
- Rarely used

Output events (kernel sets in revents field):

POLLERR - Error condition:
- Socket error occurred
- Check with `getsockopt(SO_ERROR)`
- Automatically monitored (don't set in events)

POLLHUP - Hang up:
- Connection closed by peer
- Also triggers POLLIN for final recv(0)
- Automatically monitored

POLLNVAL - Invalid request:
- FD is not open
- Usually programming error

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Common Patterns with poll() (1/2)

Pattern 1: Dynamic FD Management

```
std::vector<struct pollfd> fds;
int server_fd = create_server_socket(8080);

// Add server socket
fds.push_back({server_fd, POLLIN, 0});

while (true) {
    int ready = poll(fds.data(), fds.size(), -1);

    // Iterate backwards to safely remove elements
    for (int i = fds.size() - 1; i >= 0; i--) {
        struct pollfd& pfd = fds[i];

        if (pfd.revents & POLLIN) {
            if (pfd.fd == server_fd) {
                // Accept new connection
                int new_client = accept(server_fd, NULL, NULL);
                fds.push_back({new_client, POLLIN, 0});
            } else {
                // Handle client data
                char buffer[1024];
                int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                if (n <= 0) {
                    // Disconnect
                    close(pfd.fd);
                    fds.erase(fds.begin() + i);  // Safe: iterating backwards
                }
            }
        }

        if (pfd.revents & POLLHUP) {
            // Client disconnected
            close(pfd.fd);
            fds.erase(fds.begin() + i);
        }
    }
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Common Patterns with poll() (2/2)

Pattern 2: Monitoring Write Readiness

```
// Client wants to send large message
struct pollfd pfd;
pfd.fd = client_fd;
pfd.events = POLLIN | POLLOUT;  // Monitor both read and write
pfd.revents = 0;

// After poll()
if (pfd.revents & POLLOUT) {
    // Socket is writable - safe to send
    send(client_fd, large_data, size, 0);
}
```

Pattern 3: Checking Multiple Events

```
if (pfd.revents & POLLIN) {
    // Data available
}

if (pfd.revents & POLLOUT) {
    // Can send
}

if (pfd.revents & POLLERR) {
    int error;
    socklen_t errlen = sizeof(error);
    getsockopt(pfd.fd, SOL_SOCKET, SO_ERROR, &error, &errlen);
    std::cerr << "Socket error: " << strerror(error) << "\n";
}

if (pfd.revents & POLLHUP) {
    // Connection closed
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Performance Characteristics

Time complexity: O(n) where n = number of FDs in array

Why still O(n)?

```
// Kernel side (poll() call):
for (int i = 0; i < nfds; i++) {
    // Check if fds[i].fd is ready
    // Set fds[i].revents
}

// Userspace side (your code):
for (auto& pfd : fds) {
    if (pfd.revents & POLLIN) {
        // Handle ready FD
    }
}
```

Performance comparison:

FDs	Active	select()	poll()	Improvement
100	10	~10 μs	~8 μs	1.25x faster
500	10	~50 μs	~12 μs	4x faster
1000	10	~100 μs	~15 μs	6.7x faster
2000	10	Can't (FD_SETSIZE)	~20 μs	∞

Why poll() is faster with sparse FDs:
- select(): Must iterate 0 to max_fd (e.g., 0-5000 even if only 10 active)
- poll(): Only iterates actual FDs in array (e.g., 10 FDs)

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 When to Use poll() vs select()

Use poll() when:
- ✅ Need >1024 connections
- ✅ FD numbers are sparse (e.g., FDs 5, 100, 2000, 8000)
- ✅ Want simpler API
- ✅ Modern POSIX system (Linux, BSD, macOS)

Use select() when:
- ✅ Need microsecond timeout precision
- ✅ Old UNIX systems (select is more portable)
- ✅ Very few FDs (<100) with dense numbering
- ✅ Code simplicity more important than scalability

Use epoll() when (Topic 5):
- ✅ >1000 connections
- ✅ Need maximum performance
- ✅ Linux-only deployment acceptable

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Real-World Use Cases

1. Web proxy (thousands of connections):

```
// poll() handles 5000 concurrent HTTP connections
// No FD_SETSIZE limit
std::vector<struct pollfd> connections;
for (int i = 0; i < 5000; i++) {
    connections.push_back({client_fds[i], POLLIN | POLLOUT, 0});
}
poll(connections.data(), connections.size(), -1);
```

2. Game server (1000 players):

```
// Poll all player connections + stdin
std::vector<struct pollfd> players;
players.push_back({STDIN_FILENO, POLLIN, 0});  // Console input
for (int player_fd : active_players) {
    players.push_back({player_fd, POLLIN, 0});
}
poll(players.data(), players.size(), 16);  // 16ms for 60 FPS
```

3. Database connection pool:

```
// Monitor 200 database connections
std::vector<struct pollfd> db_pool;
for (auto& conn : database_connections) {
    db_pool.push_back({conn.fd, POLLIN, 0});
}
poll(db_pool.data(), db_pool.size(), 1000);  // 1 second timeout
```

---

poll() - Scalable I/O Multiplexing Without FD Limits

Theory

📌 Limitations of poll()

Despite improvements over select(), poll() has limitations:

1. Still O(n) complexity:
- Must iterate all FDs in array every call
- Doesn't scale well to >10,000 connections

2. Kernel must copy entire array:
- Every poll() call copies pollfd array to kernel space
- With 10,000 FDs: 10,000 * sizeof(pollfd) = 160KB copied each call

3. No edge-triggered mode:
- Always level-triggered (reports "is ready" not "became ready")
- Can't detect state transitions

4. No event registration:
- Must specify all FDs every call
- Can't "register once, poll many times"

Solution: epoll() on Linux (Topic 5) or kqueue() on BSD

---

poll() - Scalable I/O Multiplexing Without FD Limits

Edge Cases

📌 1. Large Array Performance

Problem: With 10,000 FDs, iterating the array after poll() is expensive.

```
std::vector<struct pollfd> fds(10000);

// ❌ O(10000) every time poll() returns
for (auto& pfd : fds) {
    if (pfd.revents != 0) {
        handle_events(pfd);
    }
}
```

Solution: Early exit when all ready FDs processed

```
int ready = poll(fds.data(), fds.size(), -1);
int processed = 0;

for (auto& pfd : fds) {
    if (pfd.revents != 0) {
        handle_events(pfd);
        processed++;

        if (processed >= ready) {
            break;  // ✅ All ready FDs handled, stop iterating
        }
    }
}
```

Performance impact:
- Before: Always iterate 10,000 FDs
- After: Iterate only until all ready FDs found
- If 10 ready: iterate ~10 on average (100x faster)

---

poll() - Scalable I/O Multiplexing Without FD Limits

Edge Cases

📌 2. Removing FDs During Iteration

Problem: Removing elements while iterating forward invalidates iterators.

```
// ❌ Undefined behavior
for (size_t i = 0; i < fds.size(); i++) {
    if (fds[i].revents & POLLHUP) {
        fds.erase(fds.begin() + i);  // ❌ Shifts remaining elements
        // i now points to wrong element
    }
}
```

Solution 1: Iterate backwards

```
// ✅ Safe: erasing doesn't affect indices < i
for (int i = fds.size() - 1; i >= 0; i--) {
    if (fds[i].revents & POLLHUP) {
        close(fds[i].fd);
        fds.erase(fds.begin() + i);  // Safe
    }
}
```

Solution 2: Mark for deletion, remove later

```
std::vector<int> to_remove;

for (size_t i = 0; i < fds.size(); i++) {
    if (fds[i].revents & POLLHUP) {
        close(fds[i].fd);
        to_remove.push_back(i);
    }
}

// Remove in reverse order to keep indices valid
for (auto it = to_remove.rbegin(); it != to_remove.rend(); ++it) {
    fds.erase(fds.begin() + *it);
}
```

---

poll() - Scalable I/O Multiplexing Without FD Limits

Edge Cases

📌 3. Ignoring POLLHUP

Problem: Not handling POLLHUP leaves closed connections in array.

```
// ❌ Only checks POLLIN
if (pfd.revents & POLLIN) {
    int n = recv(pfd.fd, buffer, sizeof(buffer), 0);
    // What if POLLHUP is also set?
}
```

Why it matters:
- POLLHUP and POLLIN can both be set
- Must check POLLIN first (to read final data), then POLLHUP

Solution: Check in correct order

```
// ✅ Correct order
if (pfd.revents & POLLIN) {
    int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

    if (n == 0) {
        // EOF - close connection
        close(pfd.fd);
        mark_for_removal(i);
    } else {
        process_data(buffer, n);
    }
}

// Check POLLHUP after processing POLLIN
if (pfd.revents & POLLHUP) {
    // Connection closed
    close(pfd.fd);
    mark_for_removal(i);
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Edge Cases

📌 4. Reusing pollfd Array Entries (1/2)

Problem: Setting fd to -1 tells poll() to ignore that entry.

```
// Instead of erasing (expensive), reuse slots
struct pollfd pfd = {-1, POLLIN, 0};  // ✅ poll() ignores fd < 0

// Later, reuse slot
pfd.fd = new_client;
pfd.events = POLLIN;
pfd.revents = 0;
```

Pattern: Free slot management

```
std::vector<struct pollfd> fds;
std::queue<int> free_slots;  // Indices of free slots

// Remove connection
void remove_connection(int index) {
    close(fds[index].fd);
    fds[index].fd = -1;  // Mark as free
    free_slots.push(index);
}

// Add connection
void add_connection(int new_fd) {
    if (!free_slots.empty()) {
        // ✅ Reuse free slot
        int slot = free_slots.front();
        free_slots.pop();
        fds[slot] = {new_fd, POLLIN, 0};
    } else {
        // ✅ No free slots, grow array
        fds.push_back({new_fd, POLLIN, 0});
    }
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Edge Cases

📌 4. Reusing pollfd Array Entries (2/2)

Benefits:
- Avoids expensive vector erase/shift
- Keeps array size stable
- Better cache locality

---

poll() - Scalable I/O Multiplexing Without FD Limits

Edge Cases

📌 5. Timeout Precision Loss

Problem: poll() timeout is milliseconds, not microseconds.

```
// select(): Microsecond precision
struct timeval tv = {0, 500};  // 500 microseconds
select(..., &tv);

// poll(): Millisecond precision
poll(..., 1);  // 1 millisecond (1000 microseconds)
// ❌ Can't specify 500 microseconds
```

Impact:
- High-frequency polling (e.g., game loops at 120 FPS) less precise
- 500 µs becomes 1 ms (2x slower)

Workaround: Use select() or ppoll() (Linux)

```
#include <sys/poll.h>

struct timespec ts;
ts.tv_sec = 0;
ts.tv_nsec = 500000;  // 500 microseconds in nanoseconds

ppoll(fds, nfds, &ts, NULL);  // ✅ Nanosecond precision
```

---

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 1. Basic Echo Server with poll() (1/4)

Purpose: Minimal echo server using poll() to handle multiple clients.

Concepts: pollfd array, POLLIN/POLLHUP handling, dynamic FD management.

Compilation:

Testing:

```
#include <iostream>
#include <vector>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/poll.h>
#include <netinet/in.h>
#include <arpa/inet.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

int main() {
    // Create listening socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    // Set socket options
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // Bind to port
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind");
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 1. Basic Echo Server with poll() (2/4)

```
// ...continued from previous slide
        close(server_fd);
        return 1;
    }

    // Listen
    if (listen(server_fd, 10) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
    }

    std::cout << "Server listening on port " << PORT << "\n";

    // Create pollfd array with server socket
    std::vector<struct pollfd> fds;
    fds.push_back({server_fd, POLLIN, 0});

    // Main loop
    while (true) {
        // Wait for events
        int ready = poll(fds.data(), fds.size(), -1);

        if (ready < 0) {
            perror("poll");
            break;
        }

        // Iterate backwards to safely remove disconnected clients
        for (int i = fds.size() - 1; i >= 0; i--) {
            struct pollfd& pfd = fds[i];

            // Check if this FD has events
            if (pfd.revents == 0) {
                continue;  // No events for this FD
            }

            if (pfd.revents & POLLIN) {
                if (pfd.fd == server_fd) {
                    // New connection on listening socket
                    struct sockaddr_in client_addr;
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 1. Basic Echo Server with poll() (3/4)

```
// ...continued from previous slide
                    socklen_t addr_len = sizeof(client_addr);
                    int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                    if (new_client < 0) {
                        perror("accept");
                        continue;
                    }

                    // Add to poll array
                    fds.push_back({new_client, POLLIN, 0});

                    std::cout << "New client " << new_client << " connected from "
                              << inet_ntoa(client_addr.sin_addr) << "\n";
                }
                else {
                    // Data from existing client
                    char buffer[BUFFER_SIZE];
                    int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        // Client disconnected or error
                        if (n == 0) {
                            std::cout << "Client " << pfd.fd << " disconnected\n";
                        } else {
                            perror("recv");
                        }

                        close(pfd.fd);
                        fds.erase(fds.begin() + i);
                    }
                    else {
                        // Echo back to client
                        buffer[n] = '\0';
                        std::cout << "Client " << pfd.fd << ": " << buffer;

                        send(pfd.fd, buffer, n, 0);
                    }
                }
            }

// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 1. Basic Echo Server with poll() (4/4)

[Code continues across slides]

```
// ...continued from previous slide
            // Check for hang up
            if (pfd.revents & POLLHUP) {
                std::cout << "Client " << pfd.fd << " hung up\n";
                close(pfd.fd);
                fds.erase(fds.begin() + i);
            }

            // Check for errors
            if (pfd.revents & POLLERR) {
                std::cout << "Error on FD " << pfd.fd << "\n";
                close(pfd.fd);
                fds.erase(fds.begin() + i);
            }
        }
    }

    close(server_fd);
    return 0;
}
```

```
['**Testing**:']
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 2. Chat Server with Broadcast (1/4)

Purpose: Multi-client chat where messages are broadcast to all connected clients.

Concepts: Iterating all FDs to broadcast, tracking client metadata, partial send handling.

Compilation and Testing:

Key points:
- Broadcast to all clients except sender using loop
- Track client metadata in std::map
- Use MSG_NOSIGNAL to prevent SIGPIPE on closed sockets

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/poll.h>
#include <netinet/in.h>
#include <arpa/inet.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

struct ClientInfo {
    std::string name;
    struct sockaddr_in addr;
};

int main() {
    // Create and bind server socket (same as Example 1)
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 2. Chat Server with Broadcast (2/4)

```
// ...continued from previous slide
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Chat server listening on port " << PORT << "\n";

    // Track clients and their info
    std::vector<struct pollfd> fds;
    std::map<int, ClientInfo> clients;  // fd -> client info

    fds.push_back({server_fd, POLLIN, 0});

    // Broadcast helper
    auto broadcast = [&](const std::string& message, int sender_fd) {
        for (size_t i = 1; i < fds.size(); i++) {  // Skip server_fd at index 0
            int client_fd = fds[i].fd;

            if (client_fd != sender_fd && client_fd >= 0) {
                send(client_fd, message.c_str(), message.size(), MSG_NOSIGNAL);
            }
        }
    };

    while (true) {
        int ready = poll(fds.data(), fds.size(), -1);

        if (ready < 0) {
            perror("poll");
            break;
        }

        // Process events (iterate backwards for safe removal)
        for (int i = fds.size() - 1; i >= 0; i--) {
            struct pollfd& pfd = fds[i];

            if (pfd.revents == 0) continue;

            if (pfd.revents & POLLIN) {
                if (pfd.fd == server_fd) {
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 2. Chat Server with Broadcast (3/4)

```
// ...continued from previous slide
                    // New connection
                    struct sockaddr_in client_addr;
                    socklen_t addr_len = sizeof(client_addr);
                    int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                    if (new_client >= 0) {
                        fds.push_back({new_client, POLLIN, 0});

                        // Store client info
                        ClientInfo info;
                        info.name = "User" + std::to_string(new_client);
                        info.addr = client_addr;
                        clients[new_client] = info;

                        std::string welcome = info.name + " joined the chat\n";
                        std::cout << welcome;
                        broadcast(welcome, -1);  // -1 = send to all

                        // Send welcome message to new client
                        std::string msg = "Welcome! You are " + info.name + "\n";
                        send(new_client, msg.c_str(), msg.size(), 0);
                    }
                }
                else {
                    // Data from client
                    char buffer[BUFFER_SIZE];
                    int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        // Disconnect
                        std::string name = clients[pfd.fd].name;
                        std::string goodbye = name + " left the chat\n";
                        std::cout << goodbye;
                        broadcast(goodbye, pfd.fd);

                        close(pfd.fd);
                        clients.erase(pfd.fd);
                        fds.erase(fds.begin() + i);
                    }
                    else {
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 2. Chat Server with Broadcast (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                        // Broadcast message to all other clients
                        buffer[n] = '\0';
                        std::string message = clients[pfd.fd].name + ": " + buffer;
                        std::cout << message;
                        broadcast(message, pfd.fd);
                    }
                }
            }

            if (pfd.revents & (POLLHUP | POLLERR)) {
                // Client disconnected
                if (clients.count(pfd.fd)) {
                    std::string goodbye = clients[pfd.fd].name + " disconnected\n";
                    std::cout << goodbye;
                    broadcast(goodbye, pfd.fd);
                    clients.erase(pfd.fd);
                }

                close(pfd.fd);
                fds.erase(fds.begin() + i);
            }
        }
    }

    close(server_fd);
    return 0;
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 3. Write Monitoring with POLLOUT (1/5)

Purpose: Handle partial sends using POLLOUT to monitor write readiness.

Concepts: POLLOUT event, write queues per client, dynamic event monitoring.

Compilation:

Testing:

Key points:
- Only monitor POLLOUT when data is queued
- Handle partial sends by keeping remaining data
- Non-blocking sockets required for proper EAGAIN handling

```
#include <iostream>
#include <vector>
#include <map>
#include <queue>
#include <string>
#include <cstring>
#include <unistd.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <sys/poll.h>
#include <netinet/in.h>
#include <errno.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

// Per-client write queue
std::map<int, std::queue<std::string>> write_queues;

// Helper: Set socket non-blocking
void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

// Helper: Queue data for sending
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 3. Write Monitoring with POLLOUT (2/5)

```
// ...continued from previous slide
void queue_send(int fd, const std::string& data) {
    write_queues[fd].push(data);
}

// Helper: Update pollfd events based on write queue
void update_events(std::vector<struct pollfd>& fds, int index) {
    int fd = fds[index].fd;

    if (write_queues[fd].empty()) {
        // No data to send, only monitor reads
        fds[index].events = POLLIN;
    } else {
        // Data pending, monitor reads and writes
        fds[index].events = POLLIN | POLLOUT;
    }
}

int main() {
    // Create server socket (same as before)
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Server with POLLOUT handling on port " << PORT << "\n";

    std::vector<struct pollfd> fds;
    fds.push_back({server_fd, POLLIN, 0});

    while (true) {
        int ready = poll(fds.data(), fds.size(), -1);

        if (ready < 0) {
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 3. Write Monitoring with POLLOUT (3/5)

```
// ...continued from previous slide
            perror("poll");
            break;
        }

        for (int i = fds.size() - 1; i >= 0; i--) {
            struct pollfd& pfd = fds[i];

            if (pfd.revents == 0) continue;

            // Handle new connections
            if (pfd.fd == server_fd && (pfd.revents & POLLIN)) {
                int new_client = accept(server_fd, NULL, NULL);

                if (new_client >= 0) {
                    set_nonblocking(new_client);
                    fds.push_back({new_client, POLLIN, 0});
                    write_queues[new_client] = std::queue<std::string>();

                    std::cout << "Client " << new_client << " connected\n";
                }
                continue;
            }

            // Handle client reads
            if (pfd.revents & POLLIN) {
                char buffer[BUFFER_SIZE];
                int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                if (n <= 0) {
                    // Disconnect
                    std::cout << "Client " << pfd.fd << " disconnected\n";
                    close(pfd.fd);
                    write_queues.erase(pfd.fd);
                    fds.erase(fds.begin() + i);
                }
                else {
                    // Queue large response (simulating partial sends)
                    buffer[n] = '\0';
                    std::string response = "ECHO: " + std::string(buffer);

// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 3. Write Monitoring with POLLOUT (4/5)

```
// ...continued from previous slide
                    // Simulate large response by repeating 100 times
                    std::string large_response;
                    for (int j = 0; j < 100; j++) {
                        large_response += response;
                    }

                    queue_send(pfd.fd, large_response);
                    update_events(fds, i);

                    std::cout << "Queued " << large_response.size()
                              << " bytes for client " << pfd.fd << "\n";
                }
            }

            // Handle client writes
            if (pfd.revents & POLLOUT) {
                if (!write_queues[pfd.fd].empty()) {
                    std::string& data = write_queues[pfd.fd].front();

                    // Try to send data
                    ssize_t sent = send(pfd.fd, data.c_str(), data.size(), MSG_NOSIGNAL);

                    if (sent > 0) {
                        std::cout << "Sent " << sent << " bytes to client "
                                  << pfd.fd << "\n";

                        if (static_cast<size_t>(sent) < data.size()) {
                            // Partial send - keep remaining data
                            data.erase(0, sent);
                        } else {
                            // Complete send - remove from queue
                            write_queues[pfd.fd].pop();
                        }
                    }
                    else if (sent < 0 && errno != EAGAIN && errno != EWOULDBLOCK) {
                        // Error (not EAGAIN)
                        perror("send");
                        close(pfd.fd);
                        write_queues.erase(pfd.fd);
                        fds.erase(fds.begin() + i);
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 3. Write Monitoring with POLLOUT (5/5)

[Code continues across slides]

```
// ...continued from previous slide
                    }

                    // Update events based on queue state
                    update_events(fds, i);
                }
            }

            // Handle errors
            if (pfd.revents & (POLLHUP | POLLERR)) {
                std::cout << "Client " << pfd.fd << " error/hangup\n";
                close(pfd.fd);
                write_queues.erase(pfd.fd);
                fds.erase(fds.begin() + i);
            }
        }
    }

    close(server_fd);
    return 0;
}
```

```
['**Testing**:']
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 4. Timeout-Based Operations (1/5)

Purpose: Use poll() timeout for periodic tasks (cleanup, heartbeat, stats).

Concepts: Timeout parameter, periodic work, client idle detection.

Compilation and Testing:

Key points:
- poll() timeout enables periodic tasks without separate timer
- Check for `ready == 0` to detect timeout
- Useful for cleanup, heartbeat checks, stats collection

```
#include <iostream>
#include <vector>
#include <map>
#include <cstring>
#include <ctime>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/poll.h>
#include <netinet/in.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;
const int TIMEOUT_MS = 5000;        // 5 second poll timeout
const int IDLE_TIMEOUT = 30;        // 30 seconds idle = kick

struct ClientInfo {
    time_t last_activity;
};

std::map<int, ClientInfo> clients;

void check_idle_clients(std::vector<struct pollfd>& fds) {
    time_t now = time(NULL);

    for (int i = fds.size() - 1; i >= 0; i--) {
        int fd = fds[i].fd;

        if (clients.count(fd)) {
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 4. Timeout-Based Operations (2/5)

```
// ...continued from previous slide
            int idle_seconds = now - clients[fd].last_activity;

            if (idle_seconds > IDLE_TIMEOUT) {
                std::cout << "Kicking idle client " << fd
                          << " (idle for " << idle_seconds << "s)\n";

                const char* msg = "Disconnected: idle timeout\n";
                send(fd, msg, strlen(msg), MSG_NOSIGNAL);

                close(fd);
                clients.erase(fd);
                fds.erase(fds.begin() + i);
            }
        }
    }
}

void print_stats(const std::vector<struct pollfd>& fds) {
    std::cout << "\n=== Server Stats ===\n";
    std::cout << "Active connections: " << (fds.size() - 1) << "\n";

    time_t now = time(NULL);
    for (size_t i = 1; i < fds.size(); i++) {
        int fd = fds[i].fd;
        if (clients.count(fd)) {
            int idle = now - clients[fd].last_activity;
            std::cout << "  Client " << fd << ": idle " << idle << "s\n";
        }
    }
    std::cout << "==================\n\n";
}

int main() {
    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 4. Timeout-Based Operations (3/5)

```
// ...continued from previous slide
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Server with timeout monitoring on port " << PORT << "\n";
    std::cout << "Idle timeout: " << IDLE_TIMEOUT << " seconds\n";
    std::cout << "Stats printed every " << (TIMEOUT_MS / 1000) << " seconds\n\n";

    std::vector<struct pollfd> fds;
    fds.push_back({server_fd, POLLIN, 0});

    while (true) {
        // poll() with 5-second timeout
        int ready = poll(fds.data(), fds.size(), TIMEOUT_MS);

        if (ready < 0) {
            perror("poll");
            break;
        }

        if (ready == 0) {
            // Timeout - no events, do periodic work
            std::cout << "poll() timeout - performing maintenance...\n";
            check_idle_clients(fds);
            print_stats(fds);
            continue;
        }

        // Process events
        for (int i = fds.size() - 1; i >= 0; i--) {
            struct pollfd& pfd = fds[i];

            if (pfd.revents == 0) continue;

            if (pfd.revents & POLLIN) {
                if (pfd.fd == server_fd) {
                    // New connection
                    int new_client = accept(server_fd, NULL, NULL);
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 4. Timeout-Based Operations (4/5)

```
// ...continued from previous slide

                    if (new_client >= 0) {
                        fds.push_back({new_client, POLLIN, 0});

                        ClientInfo info;
                        info.last_activity = time(NULL);
                        clients[new_client] = info;

                        std::cout << "Client " << new_client << " connected\n";
                    }
                }
                else {
                    // Client data
                    char buffer[BUFFER_SIZE];
                    int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        std::cout << "Client " << pfd.fd << " disconnected\n";
                        close(pfd.fd);
                        clients.erase(pfd.fd);
                        fds.erase(fds.begin() + i);
                    }
                    else {
                        // Update activity timestamp
                        clients[pfd.fd].last_activity = time(NULL);

                        buffer[n] = '\0';
                        std::cout << "Client " << pfd.fd << ": " << buffer;

                        send(pfd.fd, buffer, n, 0);
                    }
                }
            }

            if (pfd.revents & (POLLHUP | POLLERR)) {
                std::cout << "Client " << pfd.fd << " error/hangup\n";
                close(pfd.fd);
                clients.erase(pfd.fd);
                fds.erase(fds.begin() + i);
            }
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 4. Timeout-Based Operations (5/5)

[Code continues across slides]

```
// ...continued from previous slide
        }
    }

    close(server_fd);
    return 0;
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 5. Free Slot Reuse Pattern (1/4)

Purpose: Avoid expensive vector erase by reusing slots with fd = -1.

Concepts: Free slot management, stable indices, cache-friendly iteration.

Testing:

Benefits:
- Array size stays stable (no growth/shrink thrashing)
- No expensive erase/shift operations
- Better cache locality
- Can iterate forward (indices stable)

```
#include <iostream>
#include <vector>
#include <queue>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/poll.h>
#include <netinet/in.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

std::queue<int> free_slots;  // Indices of free pollfd slots

void remove_client(std::vector<struct pollfd>& fds, int index) {
    int fd = fds[index].fd;

    std::cout << "Removing client " << fd << " from slot " << index << "\n";

    close(fd);
    fds[index].fd = -1;  // Mark slot as free (poll() ignores fd < 0)
    fds[index].events = 0;
    fds[index].revents = 0;

    free_slots.push(index);  // Track free slot for reuse
}

// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 5. Free Slot Reuse Pattern (2/4)

```
// ...continued from previous slide
int add_client(std::vector<struct pollfd>& fds, int new_fd) {
    int slot_index;

    if (!free_slots.empty()) {
        // Reuse free slot
        slot_index = free_slots.front();
        free_slots.pop();

        fds[slot_index].fd = new_fd;
        fds[slot_index].events = POLLIN;
        fds[slot_index].revents = 0;

        std::cout << "Client " << new_fd << " added to reused slot "
                  << slot_index << "\n";
    }
    else {
        // No free slots, grow array
        slot_index = fds.size();
        fds.push_back({new_fd, POLLIN, 0});

        std::cout << "Client " << new_fd << " added to new slot "
                  << slot_index << "\n";
    }

    return slot_index;
}

int main() {
    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 5. Free Slot Reuse Pattern (3/4)

```
// ...continued from previous slide

    std::cout << "Server with free slot reuse on port " << PORT << "\n\n";

    std::vector<struct pollfd> fds;
    fds.push_back({server_fd, POLLIN, 0});

    int total_connections = 0;
    int peak_array_size = 1;

    while (true) {
        int ready = poll(fds.data(), fds.size(), -1);

        if (ready < 0) {
            perror("poll");
            break;
        }

        // Process events (no need to iterate backwards!)
        for (size_t i = 0; i < fds.size(); i++) {
            struct pollfd& pfd = fds[i];

            // Skip free slots (fd < 0)
            if (pfd.fd < 0) continue;

            if (pfd.revents == 0) continue;

            if (pfd.revents & POLLIN) {
                if (pfd.fd == server_fd) {
                    // New connection
                    int new_client = accept(server_fd, NULL, NULL);

                    if (new_client >= 0) {
                        add_client(fds, new_client);
                        total_connections++;

                        if (fds.size() > peak_array_size) {
                            peak_array_size = fds.size();
                        }

                        std::cout << "Stats: array size=" << fds.size()
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 5. Free Slot Reuse Pattern (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                                << ", free slots=" << free_slots.size()
                                << ", total clients=" << total_connections << "\n\n";
                    }
                }
                else {
                    // Client data
                    char buffer[BUFFER_SIZE];
                    int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        remove_client(fds, i);

                        std::cout << "Stats: array size=" << fds.size()
                                  << ", free slots=" << free_slots.size() << "\n\n";
                    }
                    else {
                        buffer[n] = '\0';
                        std::cout << "Client " << pfd.fd << ": " << buffer;
                        send(pfd.fd, buffer, n, 0);
                    }
                }
            }

            if (pfd.revents & (POLLHUP | POLLERR)) {
                remove_client(fds, i);

                std::cout << "Stats: array size=" << fds.size()
                          << ", free slots=" << free_slots.size() << "\n\n";
            }
        }
    }

    close(server_fd);

    std::cout << "\n=== Final Stats ===\n";
    std::cout << "Total connections served: " << total_connections << "\n";
    std::cout << "Peak array size: " << peak_array_size << "\n";

    return 0;
}
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (1/10)

Purpose: Feature-complete server with logging, metrics, graceful shutdown, connection limits.

Concepts: All production patterns combined, real-world deployment ready.

Compilation:

Running:

Testing under load:

Features:
- ✅ Graceful shutdown on SIGINT/SIGTERM
- ✅ Connection limit enforcement (MAX_CLIENTS)
- ✅ Idle client detection and cleanup
- ✅ Per-client metrics (bytes RX/TX)
- ✅ Server-wide statistics
- ✅ Timestamped logging
- ✅ Non-blocking I/O with write queues
- ✅ Free slot reuse for efficiency
- ✅ Early exit optimization (process only ready FDs)
- ✅ Production-ready error handling

🎉 **Break 1 Complete!**

Topic 4 now has:
- ✅ Comprehensive THEORY_SECTION (select() problems, poll() API, comparison, patterns, performance)
- ✅ 5 EDGE_CASES (large arrays, removal during iteration, POLLHUP handling, free slots, timeout precision)
- ✅ 6 CODE_EXAMPLES (basic echo, chat broadcast, POLLOUT handling, timeouts, free slots, production server)

Next: Break 2 will add INTERVIEW_QA section (20 comprehensive questions).

```
#include <iostream>
#include <vector>
#include <map>
#include <queue>
#include <string>
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (2/10)

```
// ...continued from previous slide
#include <cstring>
#include <ctime>
#include <csignal>
#include <unistd.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <sys/poll.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>

// Configuration
const int PORT = 8080;
const int BUFFER_SIZE = 4096;
const int POLL_TIMEOUT_MS = 1000;  // 1 second
const int MAX_CLIENTS = 1000;
const int IDLE_TIMEOUT = 60;       // 60 seconds

// Metrics
struct ServerMetrics {
    uint64_t total_connections = 0;
    uint64_t total_bytes_received = 0;
    uint64_t total_bytes_sent = 0;
    uint64_t current_connections = 0;
    uint64_t rejected_connections = 0;
    time_t start_time = time(NULL);
};

ServerMetrics metrics;

// Per-client state
struct ClientState {
    std::string remote_addr;
    time_t connected_at;
    time_t last_activity;
    uint64_t bytes_received = 0;
    uint64_t bytes_sent = 0;
    std::queue<std::string> write_queue;
};

// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (3/10)

```
// ...continued from previous slide
std::map<int, ClientState> clients;

// Global state
volatile sig_atomic_t keep_running = 1;
std::queue<int> free_slots;

// Signal handler for graceful shutdown
void handle_signal(int sig) {
    if (sig == SIGINT || sig == SIGTERM) {
        std::cout << "\nReceived signal " << sig << ", shutting down...\n";
        keep_running = 0;
    }
}

// Logging helpers
void log_info(const std::string& msg) {
    time_t now = time(NULL);
    char timestamp[32];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", localtime(&now));
    std::cout << "[" << timestamp << "] [INFO] " << msg << "\n";
}

void log_error(const std::string& msg) {
    time_t now = time(NULL);
    char timestamp[32];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", localtime(&now));
    std::cerr << "[" << timestamp << "] [ERROR] " << msg << "\n";
}

// Set socket non-blocking
void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags == -1) flags = 0;
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

// Update pollfd events based on write queue
void update_poll_events(std::vector<struct pollfd>& fds, int index) {
    int fd = fds[index].fd;

// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (4/10)

```
// ...continued from previous slide
    if (clients[fd].write_queue.empty()) {
        fds[index].events = POLLIN;
    } else {
        fds[index].events = POLLIN | POLLOUT;
    }
}

// Remove client
void remove_client(std::vector<struct pollfd>& fds, int index) {
    int fd = fds[index].fd;

    if (clients.count(fd)) {
        ClientState& client = clients[fd];

        log_info("Client " + std::to_string(fd) + " (" + client.remote_addr +
                 ") disconnected. RX: " + std::to_string(client.bytes_received) +
                 " bytes, TX: " + std::to_string(client.bytes_sent) + " bytes");

        clients.erase(fd);
        metrics.current_connections--;
    }

    close(fd);
    fds[index].fd = -1;
    fds[index].events = 0;
    fds[index].revents = 0;
    free_slots.push(index);
}

// Add new client
int add_client(std::vector<struct pollfd>& fds, int new_fd, const std::string& addr) {
    int slot_index;

    if (!free_slots.empty()) {
        slot_index = free_slots.front();
        free_slots.pop();
        fds[slot_index] = {new_fd, POLLIN, 0};
    } else {
        slot_index = fds.size();
        fds.push_back({new_fd, POLLIN, 0});
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (5/10)

```
// ...continued from previous slide
    }

    ClientState client;
    client.remote_addr = addr;
    client.connected_at = time(NULL);
    client.last_activity = client.connected_at;
    clients[new_fd] = client;

    metrics.total_connections++;
    metrics.current_connections++;

    log_info("Client " + std::to_string(new_fd) + " (" + addr + ") connected");

    return slot_index;
}

// Check idle clients and kick them
void check_idle_clients(std::vector<struct pollfd>& fds) {
    time_t now = time(NULL);

    for (size_t i = 1; i < fds.size(); i++) {
        int fd = fds[i].fd;

        if (fd >= 0 && clients.count(fd)) {
            int idle = now - clients[fd].last_activity;

            if (idle > IDLE_TIMEOUT) {
                log_info("Kicking idle client " + std::to_string(fd) +
                         " (idle for " + std::to_string(idle) + "s)");

                const char* msg = "Disconnected: idle timeout\n";
                send(fd, msg, strlen(msg), MSG_NOSIGNAL);

                remove_client(fds, i);
            }
        }
    }
}

// Print server stats
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (6/10)

```
// ...continued from previous slide
void print_stats() {
    time_t now = time(NULL);
    int uptime = now - metrics.start_time;

    std::cout << "\n" << std::string(60, '=') << "\n";
    std::cout << "SERVER STATISTICS\n";
    std::cout << std::string(60, '=') << "\n";
    std::cout << "Uptime:              " << uptime << " seconds\n";
    std::cout << "Current connections: " << metrics.current_connections << "\n";
    std::cout << "Total connections:   " << metrics.total_connections << "\n";
    std::cout << "Rejected:            " << metrics.rejected_connections << "\n";
    std::cout << "Bytes received:      " << metrics.total_bytes_received << "\n";
    std::cout << "Bytes sent:          " << metrics.total_bytes_sent << "\n";
    std::cout << std::string(60, '=') << "\n\n";
}

int main() {
    // Install signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGPIPE, SIG_IGN);  // Ignore SIGPIPE

    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        log_error("socket() failed: " + std::string(strerror(errno)));
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        log_error("bind() failed: " + std::string(strerror(errno)));
        close(server_fd);
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (7/10)

```
// ...continued from previous slide
        return 1;
    }

    if (listen(server_fd, 128) < 0) {
        log_error("listen() failed: " + std::string(strerror(errno)));
        close(server_fd);
        return 1;
    }

    log_info("Production server started on port " + std::to_string(PORT));
    log_info("Max clients: " + std::to_string(MAX_CLIENTS));
    log_info("Idle timeout: " + std::to_string(IDLE_TIMEOUT) + " seconds");

    std::vector<struct pollfd> fds;
    fds.push_back({server_fd, POLLIN, 0});

    time_t last_stats_time = time(NULL);

    while (keep_running) {
        int ready = poll(fds.data(), fds.size(), POLL_TIMEOUT_MS);

        if (ready < 0) {
            if (errno == EINTR) continue;  // Interrupted by signal
            log_error("poll() failed: " + std::string(strerror(errno)));
            break;
        }

        // Timeout - do periodic work
        if (ready == 0) {
            check_idle_clients(fds);

            time_t now = time(NULL);
            if (now - last_stats_time >= 60) {
                print_stats();
                last_stats_time = now;
            }
            continue;
        }

        // Process ready FDs
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (8/10)

```
// ...continued from previous slide
        int processed = 0;
        for (size_t i = 0; i < fds.size() && processed < ready; i++) {
            struct pollfd& pfd = fds[i];

            if (pfd.fd < 0 || pfd.revents == 0) continue;

            processed++;

            // Handle new connections
            if (pfd.fd == server_fd && (pfd.revents & POLLIN)) {
                struct sockaddr_in client_addr;
                socklen_t addr_len = sizeof(client_addr);
                int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                if (new_client >= 0) {
                    if (metrics.current_connections >= MAX_CLIENTS) {
                        // Reject connection
                        const char* msg = "503 Server Full\r\n";
                        send(new_client, msg, strlen(msg), 0);
                        close(new_client);

                        metrics.rejected_connections++;
                        log_info("Rejected connection (server full)");
                    } else {
                        set_nonblocking(new_client);
                        std::string addr = inet_ntoa(client_addr.sin_addr);
                        add_client(fds, new_client, addr);
                    }
                }
                continue;
            }

            // Handle client reads
            if (pfd.revents & POLLIN) {
                char buffer[BUFFER_SIZE];
                int n = recv(pfd.fd, buffer, sizeof(buffer), 0);

                if (n <= 0) {
                    remove_client(fds, i);
                } else {
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (9/10)

```
// ...continued from previous slide
                    clients[pfd.fd].last_activity = time(NULL);
                    clients[pfd.fd].bytes_received += n;
                    metrics.total_bytes_received += n;

                    // Echo response
                    std::string response(buffer, n);
                    clients[pfd.fd].write_queue.push(response);
                    update_poll_events(fds, i);
                }
            }

            // Handle client writes
            if (pfd.revents & POLLOUT) {
                if (!clients[pfd.fd].write_queue.empty()) {
                    std::string& data = clients[pfd.fd].write_queue.front();

                    ssize_t sent = send(pfd.fd, data.c_str(), data.size(), MSG_NOSIGNAL);

                    if (sent > 0) {
                        clients[pfd.fd].bytes_sent += sent;
                        metrics.total_bytes_sent += sent;

                        if (static_cast<size_t>(sent) < data.size()) {
                            data.erase(0, sent);
                        } else {
                            clients[pfd.fd].write_queue.pop();
                        }

                        update_poll_events(fds, i);
                    }
                    else if (sent < 0 && errno != EAGAIN && errno != EWOULDBLOCK) {
                        log_error("send() failed for client " + std::to_string(pfd.fd) +
                                  ": " + strerror(errno));
                        remove_client(fds, i);
                    }
                }
            }

            // Handle errors
            if (pfd.revents & (POLLHUP | POLLERR)) {
// ...continues on next slide
```

poll() - Scalable I/O Multiplexing Without FD Limits

Code Examples

📌 6. Production-Grade poll() Server (10/10)

[Code continues across slides]

```
// ...continued from previous slide
                remove_client(fds, i);
            }
        }
    }

    // Graceful shutdown
    log_info("Shutting down gracefully...");

    // Close all client connections
    for (size_t i = 1; i < fds.size(); i++) {
        if (fds[i].fd >= 0) {
            close(fds[i].fd);
        }
    }

    close(server_fd);

    // Print final stats
    print_stats();

    log_info("Server shutdown complete");

    return 0;
}
```

```
['**Running**:', '**Testing under load**:']
```

What You Learned

Key Takeaways

- ✓ The Problem with select()
- ✓ What is poll()?
- ✓ The pollfd Structure
- ✓ Step by Step

→ Swipe to learn →